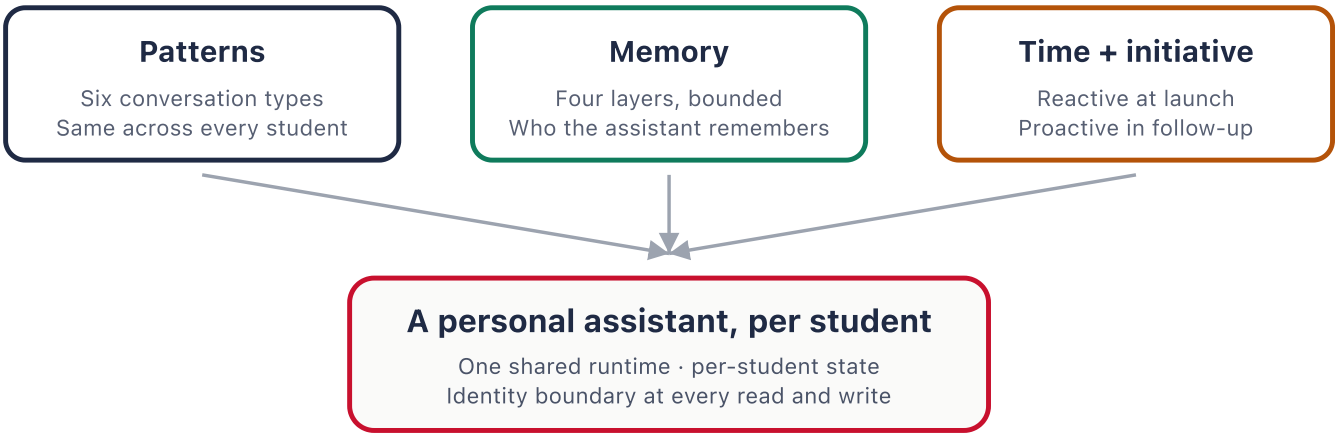
How the assistant is personalised for each student

One assistant, scoped to each student. Six conversation patterns, shaped by what the assistant remembers about them, deepening over time. The same architecture across every student; what differs is the per-student state inside it.

Sources: **Charlie Pattern Architecture v0.5** · **Workshop Briefing v1.2** · 11 May 2026

THE THREE FORCES — WHAT MAKES A PA, PER STUDENT



WHAT THIS MEANS IN PRACTICE

Patterns

Inform-and-Act, Status-Check, Decide, Express, Nudge, Check-in. Four at launch, two proactive in a follow-up release.

Memory

Bounded explicit profile + bounded chosen memory + searchable session log + implicit model. Hermes-shape; small on purpose.

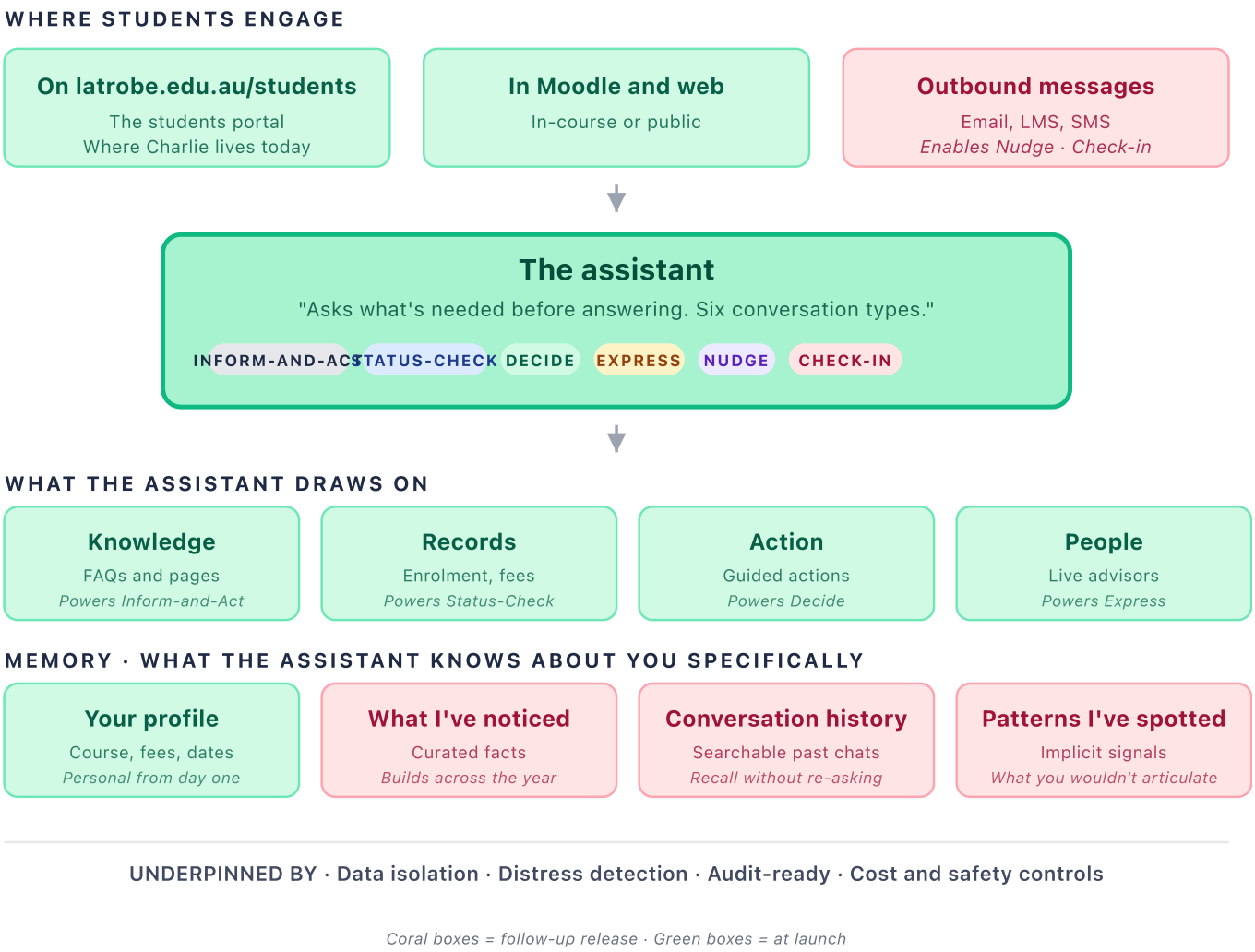
Time + initiative

Same six patterns, deepening across the year. Outbound + scheduler unlock Nudge and Check-in once persistent state ships.

> *Same six patterns throughout. What changes is depth and initiative — from session-bounded at launch to relationship-bounded in the follow-up release. — Workshop Briefing v1.2*

Solution architecture, in plain English

The same architecture handles all six conversation types. Most of it ships at launch; one slice arrives in a follow-up release. Below: the full diagram. Channels → the assistant → what it draws on → memory → underpinned by trust foundations.



UNDERPINNED BY · Data isolation · Distress detection · Audit-ready · Cost and safety controls

Coral boxes = follow-up release · Green boxes = at launch

How each conversation pattern works

Six reusable types. Same shape every time: **trigger** kicks the pattern off → **behaviour** defines what the assistant does → **routing** connects to the capability that serves it. Four reactive (student initiates), two proactive (system initiates, follow-up release).

Inform-and-Act

REACTIVE

TRIGGER
Student asks a question.

BEHAVIOUR
Probe purpose before delivering content. Surface query often hides the real need.

ROUTES TO
Knowledge (FAQs and pages, surfaced intelligently).

EXAMPLE
"What is SSAF?" → understand / pay / defer / sanction → routed answer.

Status-Check

REACTIVE

TRIGGER
Student asks where things stand.

BEHAVIOUR
Look up *their* record, reply with the answer — not a generic article.

ROUTES TO
Records (enrolment, fees, services).

EXAMPLE
"Did my course transfer go through?" → record lookup → status + next steps.

Decide

REACTIVE

TRIGGER
Student wants to take a consequential action.

BEHAVIOUR
Multi-turn flow — eligibility check → confirm → lodge. In-chat form, no link-out.

ROUTES TO
Action (the agent-callable action tools).

EXAMPLE
"I want to defer my SSAF" → eligibility → in-chat form → confirmation lodged.

Express

REACTIVE

TRIGGER
Student discloses distress, frustration, or wellbeing concern.

BEHAVIOUR
Listen, surface support, hand off in-channel to a wellbeing advisor — not a phone-number redirect.

ROUTES TO
People (live advisors).

EXAMPLE
"I'm not coping" → empathetic response → support options → held handoff state.

Nudge

PROACTIVE · LATER

TRIGGER
System-initiated — deadline or status signal.

BEHAVIOUR
Outbound message reaches the student before the deadline. Click-through into chat *already in flight*.

ROUTES TO
Outbound channel (email / LMS / SMS critical-only) → into Decide.

EXAMPLE
Email: *"You can defer your SSAF — deadline in 6 days. [Open in chat]"*.

Check-in

PROACTIVE · LATER

TRIGGER
System-initiated — behavioural at-risk signal.

BEHAVIOUR
Gentle outbound message opens a no-pressure conversation. May transition into Express.

ROUTES TO
Outbound channel → into a wellbeing-aware conversation.

EXAMPLE
LMS announcement: *"We've noticed you haven't logged into Moodle for two weeks. Is everything okay?"*

> *Every conversation fits one or more of these types, regardless of the topic. The patterns are the conversation shapes; topics flow through them. — Pattern Architecture v0.5*

Structured + unstructured repositories

The architecture uses both — each capability uses the store shape natural to its data. Structured stores for identity, records, transactional actions and audit; unstructured for knowledge, memory and accreted history. Pattern-tagged logging is the hybrid layer running across both.

STRUCTURED

Schemas · rows · exact queries

Identity
`student_id`, roles, session tokens — identity provider

Records
SIS, payments, RightNow incidents

Action APIs
The 5 transactional flows — defer, withdraw, transfer, change load, special consideration

Budgets
Per-student token counters, replenished on schedule

Audit log
Tagged events — student_id · pattern · topic · purpose · outcome

Scheduler / signals
Eligibility queries + behavioural signals (LMS engagement, missed appointments)

Risk classifier output
Severity + signal type per turn — governs Express routing

UNSTRUCTURED / SEMI-STRUCTURED

Text · documents · snippets

Knowledge base
FAQs + LTU students portal + Handbook · indexed periodically · vector + lexical

MEMORY
Agent-curated facts · markdown · bounded ~800 tokens (Hermes-style)

Session log
Conversation history · text + metadata · FTS5-style searchable

Skill documents
SKILL.md style — YAML frontmatter + markdown per scenario

Implicit user model
Learned preferences, rhythms, signals — external provider (Honcho-style)

Outbound templates
Email / LMS / SMS content with personalisation slots — follow-up release

Hybrid · pattern-tagged logging Structured tags (pattern · topic · purpose · risk · identity · lifecycle · outcome) wrapped around unstructured turn content. The analytics seam — *"32% of SSAF queries were deferral-purpose"* comes from here. Multi-tenant safe by construction; row-level security at the data tier.

PER PATTERN, WHAT GETS TOUCHED

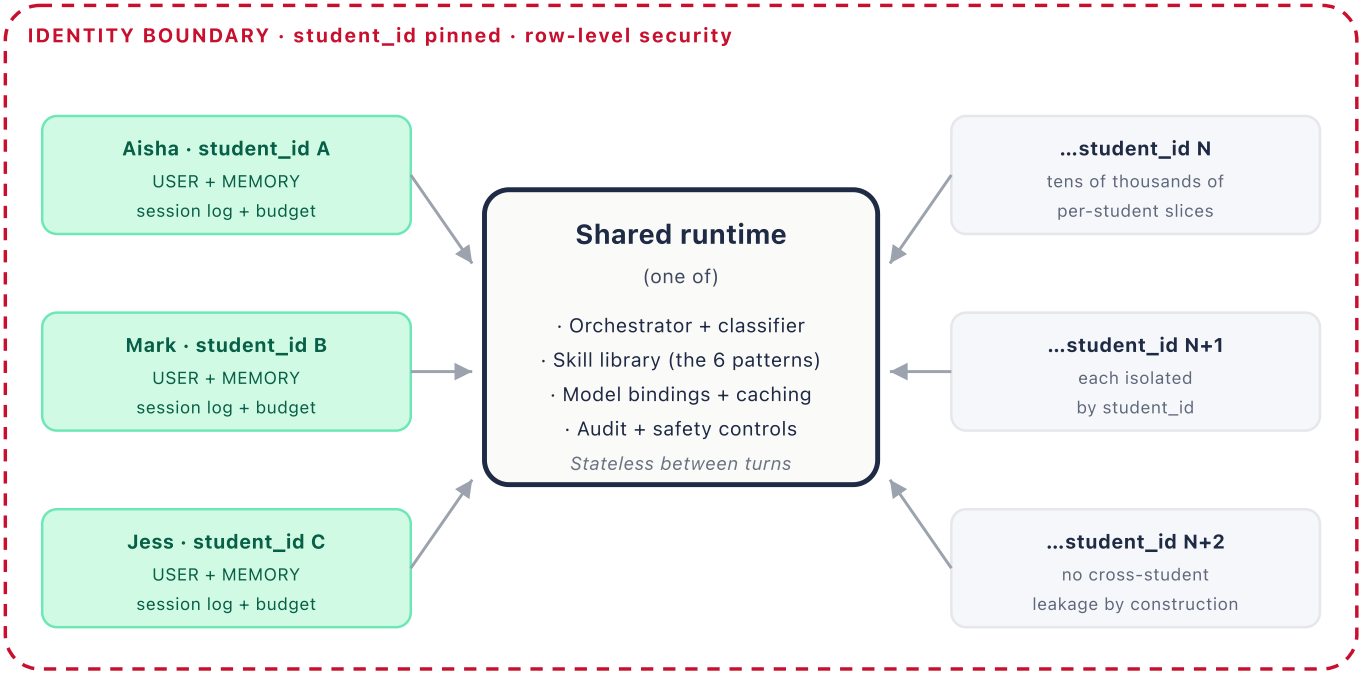
PATTERN	STRUCTURED	UNSTRUCTURED
INFORM-AND-ACT	identity · audit	Knowledge · MEMORY
STATUS-CHECK	Records · audit	MEMORY (for context)
DECIDE	Action APIs · Records · audit	MEMORY
EXPRESS	Risk classifier · People handoff · audit	MEMORY · Session log
NUDGE	Scheduler · Records · budget · audit	MEMORY · Outbound templates
CHECK-IN	Behavioural signals · audit	Implicit model · MEMORY · Outbound templates

In the prototype today, only the structured side is mocked — `StudentRecord` per persona in `src/data/student-records.ts`, scripted scenarios in `src/data/scenarios.ts`. No unstructured corpus, no real knowledge base, no agent memory writes yet (the brief-override scope items would change that). The production version pulls from both.

How each PA instance works

Conceptually there's one PA per student. **Architecturally** there's one shared runtime and many per-student state slices — that's what scales. Each student's slice is keyed by their identity; the runtime composes their context fresh for every turn.

ONE RUNTIME, MANY PER-STUDENT STATE SLICES



What's per-student (one each)

- **USER profile** — course, enrolment, fees, key dates. From SIS.
- **MEMORY** — agent-curated facts. Bounded markdown.
- **Session log** — searchable conversation history.
- **Token budget** — cost cap per period.
- **Implicit model** — preferences, rhythms.

What's shared (one of)

- **Orchestrator** — runs the pattern + topic + purpose classifier.
- **Skill library** — the six patterns, the scenarios behind them.
- **Model bindings** — the foundation model + retrieval components.
- **Safety controls** — distress hold, audit log, multi-tenancy enforcement.
- **Autonomous Curator** — periodic skill / memory review.

PER-TURN LIFECYCLE

When a student opens the assistant, the runtime composes their context, processes the turn, writes back, and is free for the next student. The "instance" is the shared runtime + that student's state slice + the active conversation context — not a long-lived process.

1. **Identify** — pull `student_id` from the auth token (identity provider → gateway → orchestrator).
2. **Load slice** — read USER + MEMORY + active conversation tail. Everything scoped to this student_id.
3. **Classify** — pattern + topic + purpose from the orchestrator. Pick the active skill.
4. **Retrieve** — past turns matched by relevance / recency from the session log if the conversation needs them.
5. **Compose + respond** — assemble the prompt, call the model, return the reply + chips.
6. **Write back** — any new MEMORY note, append to the session log, decrement the budget, write audit entries.

Memory turns six patterns into personalisation

Four layers, bounded by design. Three switch on in the follow-up release; the fourth — your profile — is personal from day one. The principle is small-by-default: explicit profile + chosen memory stay in every prompt; the rest is retrieved on demand.

THE FOUR MEMORY LAYERS

Your profile · always loaded
Course · enrolment · fees · key dates · journey overlay
~500 tokens

What I've noticed · always loaded
Agent-curated facts the assistant has chosen to remember
~800 tokens

Conversation history · retrieved on demand
FTS5-style search across past chats; only matched snippets enter
unbounded

Patterns I've spotted · adaptive
Implicit model — preferences, rhythms, signals (Honcho-style)
external provider

Pattern from Hermes Agent (Nous Research). Bounded markdown + session log + pluggable implicit model. **Forward-looking:** the memory layer can upgrade to a connected graph — cascade retrieval over tag / cluster / semantic-link edges — when the deepening-model concept matures (jcode pattern). Same architectural seam, deeper inference.

HOW EACH PATTERN IS PERSONALISED BY MEMORY

Pattern	With memory
INFORM-AND-ACT	Probe shortcuts — "you usually ask about SSAF around census, jump to deferral?"
STATUS-CHECK	Personal + contextual — "last time you were worried about the May deadline."
DECIDE	Remembers preferences — "last time you deferred; same again?"
EXPRESS	Hot start — "you mentioned feeling overwhelmed in week 6; has that come back?"
NUDGE	Targeted — only nudges those for whom the deadline is relevant.
CHECK-IN	Triggered by behavioural signals only memory can hold.

HOW IT LEARNS · THREE FEEDBACK LOOPS

Per-student loop

Assistant writes agent-curated facts into MEMORY after meaningful turns. Shapes future turns for that student only.

Per-skill loop

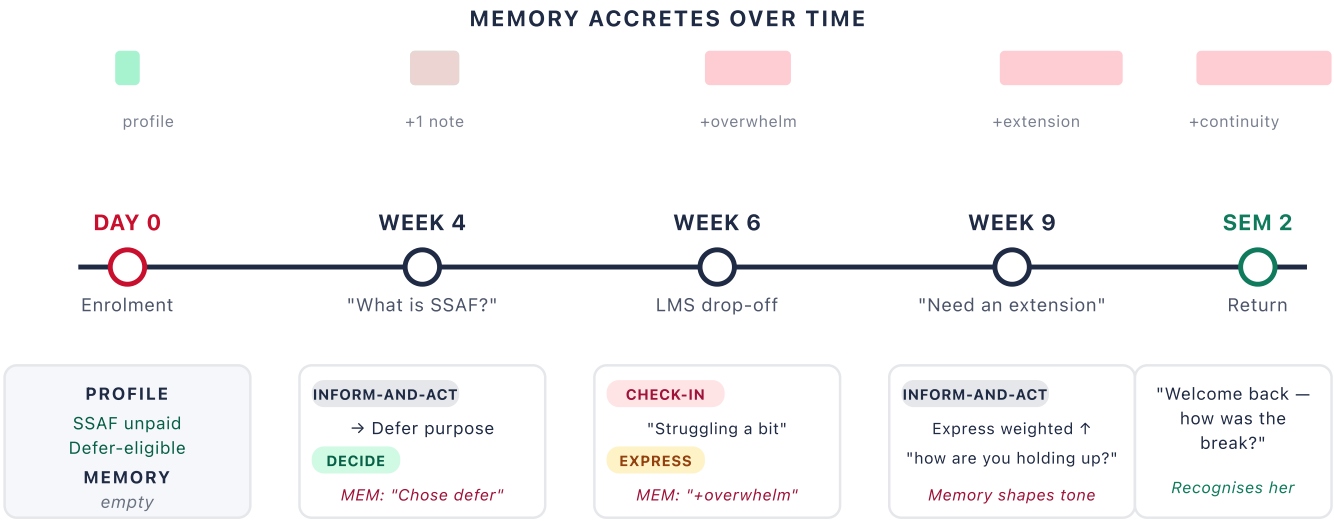
Skills (scenarios) get patched when outcomes show they're outdated. Autonomous Curator consolidates + archives.

Population loop

Pattern-tagged logging surfaces signal — *"32% of SSAF queries were deferral-purpose"* — feeding upstream fixes.

Aisha's semester — same six patterns, deepening over time

International undergraduate, Semester 1. SSAF unpaid, eligible to defer to HELP, deadline 14 May. We follow one student to show how memory accretes and the patterns shape differently as the assistant carries more.



What changes session-to-session

- **Day 0 → W4:** MEMORY: empty → 1 note (deferral preference)
- **W4 → W6:** Memory now carries Decide preference + active SSAF state
- **W6 → W9:** Express classifier weighting changes after overwhelm note
- **Sem 1 → Sem 2:** Memory persists; assistant recognises her on return

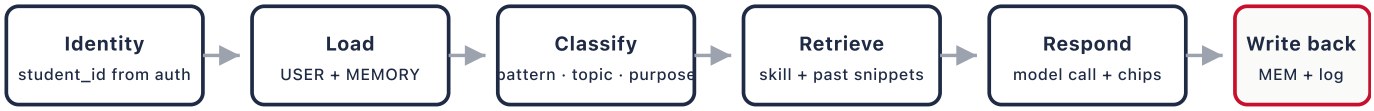
What stays the same

- Same six patterns, every turn — no new types invented per student
- Same architecture — Channels → Assistant → Capabilities → Memory → Foundations
- Same safety controls — distress hold, audit, multi-tenancy at the auth boundary
- Same identity boundary — every read and write tagged to Aisha

How context is controlled — and how it stays safe at scale

Per-turn context assembly, the four levers that keep it bounded, the safety overrides, and what changes between launch and the follow-up release.

HOW CONTEXT IS ASSEMBLED PER TURN



Every step constrained to this student_id — multi-tenant safety by construction

USER + MEMORY + system + skill instructions cached → ~80% prompt-cache hits per session

FOUR LEVERS CONTROL CONTEXT

1. **Always-in** — system prompt + USER + MEMORY + active skill. Bounded by design.
2. **Retrieved on demand** — past turns by relevance / recency.
3. **Never-in** — other students' state. `student_id` pinned at auth.
4. **Compressed** when the window fills — first 3 + last 20 protected.

SAFETY · PER STUDENT

- **Two-tier model** — auto-allowed for safe internal actions; *requires-permission* for anything that leaves the sandbox (outbound messages, advisor handoff, cross-student data writes). Persistent review queue.
- **Token budget** — hard cap on cost per student per period; treated as a regression budget — measurable, reviewable, intentionally justified.
- **Distress hold** — Express signal pauses outbound budget.
- **Audit log** — every read and write tagged with `student_id`, pattern, topic, outcome.
- **Multi-tenancy** at the auth boundary — identity provider → gateway → orchestrator.

LAUNCH VS FOLLOW-UP RELEASE · WHAT CHANGES

At launch

· 4 reactive patterns (Inform-and-Act, Status-Check, Decide, Express)
· Session-bounded — chat starts fresh each time
· USER profile always there (course, enrolment)
· Same architecture, same boundaries
· Pattern classification + per-student budgets live

A solid replacement for Charlie, working on every channel

In a follow-up release

· 6 patterns — adds Nudge + Check-in
· Relationship-bounded — memory persists
· Agent-curated facts + searchable history
· Implicit model — preferences and rhythms
· Self-scheduling agent + outbound channel + signal eval
· Autonomous Curator consolidates skills + memory

The PA framing made real — depth and initiative

> *Stakeholders have asked for an assistant that's like a personal assistant for each student. The six conversation types are the concrete specification of what "like a PA" actually means. — Workshop Briefing v1.2*

How the assistant finds what it needs

Charlie today does *RAG* — dense vector search over FAQ content. That's *one* retrieval technique. The new assistant classifies the pattern first, then routes to the retrieval method that fits: **hybrid** for knowledge questions, **structured queries** for the student's own data, **function calls** for actions.

> **Knowledge scope** — curated FAQs + the LTU students portal (`latrobe.edu.au/students`) + `handbook.latrobe.edu.au`, indexed periodically. Live page fetch only via explicit tool call when an indexer-missed URL is needed. **Not open web** — grounded by design.

THE RETRIEVAL MENU

Lexical · BM25 · FTS5

Exact terms, stems, filters. Cheap (~5ms). Hermes' session-log default. Misses paraphrase.

Dense vector

Semantic similarity. Handles intent and synonyms (~50ms). *Charlie's current RAG lives here.* Misses exact policy names.

Hybrid + rerank

BM25 + vector + RRF or cross-encoder rerank (~100-300ms). Best precision/recall — production standard for knowledge corpora.

Structured (SQL / API)

Typed, exact, transactional (~5-20ms). Right tool for records, fees, status — not RAG.

Function / tool calling

LLM picks a defined action with a typed schema. Decide pattern's natural fit — auditable, deterministic.

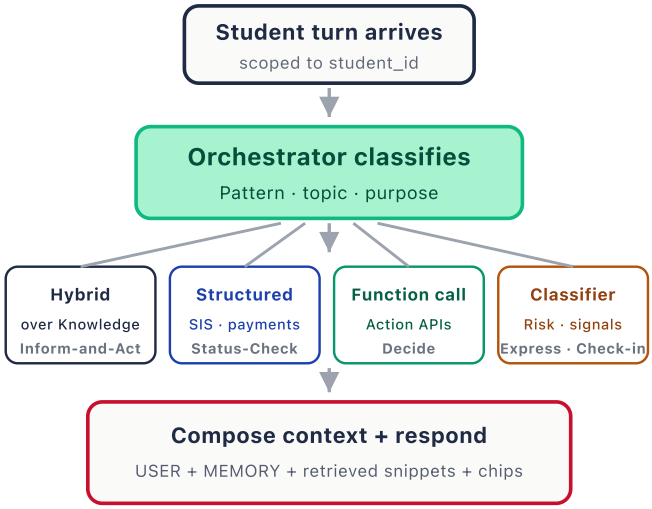
Graph queries

Entity relationships — *"who coordinates X?"*. Niche; setup overhead. Useful if a knowledge graph already exists.

PER-PATTERN ROUTING

PATTERN	FIRST RETRIEVAL STEP	THEN / FALLBACK
INFORM-AND-ACT	Hybrid over Knowledge	+ MEMORY context
STATUS-CHECK	Structured API on Records	+ Conversation history
DECIDE	Function call → Action API	+ Records (eligibility)
EXPRESS	Risk classifier (model)	+ MEMORY · handoff route
NUDGE	Scheduler signal + Records query	+ Outbound templates
CHECK-IN	Behavioural signal + Implicit model	+ Outbound templates

PATTERN CLASSIFICATION HAPPENS FIRST



CHARLIE TODAY VS THE NEW ASSISTANT

Charlie today

- Dense-vector RAG over FAQ content
- Same retrieval regardless of student intent
- No structured lookups against Records
- No purpose probe before retrieval

Misses exact policy names AND misses real student need

New assistant

- Pattern classified BEFORE retrieval
- Hybrid (BM25 + vector + rerank) over Knowledge
- Structured API on Records — exact, transactional
- Function calls for Action — auditable, deterministic

Right tool for the right question — every time

Cost shape at LTU scale: BM25 ~5ms, dense ~50ms, hybrid + rerank ~3–5x cost of lexical-only. Per-student token budgets and prompt caching (USER + MEMORY + skill instructions cached across turns) keep cost predictable per student per period.

What to build first — and why

Replacing Charlie's RAG isn't the biggest lever for LTU. Of the 11 scenarios in this brief, ~8 touch structured stores and ~4 touch the Knowledge corpus. The build priority follows that ratio: get the routing right and the structured integrations in place; the retrieval upgrade follows.

THE FOUR PHASES · IN ORDER

1 Action API integration

Rebuild the 5 transactional flows as agent-callable APIs — defer SSAF · withdraw · transfer course · change load · special consideration.

Pattern served: Decide
Scenarios unlocked: 3 of 12 (Aisha SSAF defer, Aisha SSAF nudge, Jess sanction recovery)
Why first: Unlocks "do it in the chat" — the central UX shift students feel immediately

Direct opportunity: 7,648 SSAF sanctions in Sem 1 — preventable when deferral lodges in-chat before the deadline.

2 Structured Records integration

Read-only APIs for SIS · payments · RightNow incidents · course history.

Pattern served: Status-Check · eligibility for Decide & Nudge · context for Express
Scenarios unlocked: 8–9 of 12 — most of the matrix
Why second: Every personalised answer ("did your X go through", "you're eligible to defer") depends on this

Charlie failure mode: tries to RAG "what is course transfer" instead of looking up this student's transfer.

3 Hybrid retrieval over Knowledge

Docling (parse) + LlamaIndex (orchestrate) + Qdrant (BM25 + vector, RRF fusion) + BGE reranker (precision). Replaces Charlie's dense-only RAG, correctly.

Pattern served: Inform-and-Act · the explainer half of Express
Scenarios unlocked: ~4 of 12 (SSAF Q&A, extension Q&A, supplementary policy, support resources)
Why third: Only valuable AFTER #1 + #2 — otherwise students still get RAG'd for record questions

4 Contextual Retrieval boost (optional)

Anthropic's prefix-each-chunk-with-context pattern. ~49% reported retrieval-failure reduction. Prompt caching keeps cost flat.

Pattern served: Refines #3 — precision lift only
Scenarios unlocked: Higher accuracy on the ~4 from phase 3
Why fourth: Only worth it once #3 has a measurable baseline of failure modes to fix

> *Charlie's biggest failure isn't bad RAG — it's RAG-where-RAG-doesn't-belong. The new assistant gets the routing right first; the retrieval upgrade follows.*

The first vertical slice

Don't build the architecture horizontally — layer by layer across all six patterns. Build vertically — *one slice through every layer, end-to-end*. The slice is the **SSAF deferral arc**. Once it works, the rest replicates. Sequence is by dependency, not calendar.

WHY THE SSAF DEFERRAL ARC AS THE PILOT

Attributable impact

7,648 fee-non-payment sanctions in Sem 1, most assumed SSAF. Saves something the institution can already count.

Priority coverage

Touches build phases 1, 2 and 3 at narrow scope — Action API + Records read + Hybrid retrieval — in one slice.

Bounded corpus

SSAF is a tight subset of LTU content. Indexable without indexing the whole portal. Failure modes are visible.

THE BUILD SEQUENCE · DEPENDENCY-ORDERED

1 **Orchestrator skeleton · pattern classifier · identity boundary**
Unlocks: pinned `student_id` at every read and write; pattern + topic + purpose classification routes the rest. Nothing downstream works without it.

2 **SSAF defer-to-HELP action API + Records read**
Unlocks: Decide pattern lodges the action in-chat. Reads SIS for current SSAF status and eligibility. Status-Check pattern starts working too.

3 **SSAF knowledge subset · indexed and retrievable**
Unlocks: Inform-and-Act probe routes to actual content. Hybrid retrieval (BM25 + vector + rerank) over SSAF FAQs + Handbook entries. The purpose-probe behaviour becomes observable.

4 **Internal alpha · audit, budget, distress passthrough**
Unlocks: audit log carries pattern + topic + purpose + outcome per turn. Per-student token budget counter is live. Distress classifier wired (passthrough at first; full Express handoff comes later).

5 **Measure**
Validates: did the assistant route correctly? Did eligibility fire live? Did deferrals lodge? What's the completion rate vs the Charlie baseline (essentially zero today)?

PRE-DECISIONS · BEFORE STEP 1

- **Orchestrator** — LangGraph (Python) or custom TS
- **Model** — small fast model for classification, larger for content. Per-task via `models.config.ts`. **Provider is fully swappable:** Anthropic Claude · OpenAI GPT-4o · Azure OpenAI AU · Google Gemini · GLM (Zhipu) · Mistral · AWS Bedrock · self-hosted (Ollama/vLLM). No orchestrator code changes needed to switch — `LLMClient` abstraction decouples the provider.
- **Vector store** — Qdrant (cleanest hybrid) or pgvector if Postgres already in stack
- **Hosting** — Azure (LTU's stack) — App Service or Functions
- **Identity** — Entra · `student_id` pinned at gateway

SUCCESS LOOKS LIKE

- Aisha-type scenario completes deferral in chat — without leaving the conversation
- Live SIS eligibility check fires (not mocked)
- Audit log carries `student_id` + pattern + topic + outcome on every turn
- Measurable lift in SSAF deferral completion vs Charlie baseline
- Distress signals route safely — even though full handoff comes later

Out of scope for this slice: the other 5 patterns · proactive Nudge / Check-in · outbound channels · memory persistence across sessions · self-improving skills · the other 4 action APIs. Iterations 2+, prioritised per page 10.

How each pattern sounds — the prompts behind the voice

Each pattern has its own prompt — different jobs need different tone, control flow, and success criteria. One prompt can't serve all six. Below: each pattern's job, a sketch of its system prompt, and a characteristic phrase the assistant says when that prompt is active.

Inform-and-Act

REACTIVE · PROBE

JOB Probe purpose before delivering content.

> *Generate 2–4 chip options that disambiguate. Phrase chips as the student's intent. Never deliver a topical answer until purpose is clear.*

VOICE *"Happy to help — are you trying to understand SSAF, pay it, defer it, or sort out a sanction?"*

Status-Check

REACTIVE · CHECK

JOB Look up their record. Reply with their answer.

> *Call Records for the relevant entity. Respond with their specific data — never a generic article. Include the expected next step.*

VOICE *"Your transfer application is with the Faculty for review; decision expected by 12 May."*

Decide

REACTIVE · CONFIRM

JOB Eligibility → confirm → lodge.

> *Confirm eligibility, present consequences plainly, confirm intent explicitly, invoke the Action tool. Never pre-fill without confirmation.*

VOICE *"You're eligible to defer SSAF; deadline 14 May. Ready to lodge it now? About three minutes."*

Express

REACTIVE · OFFER

JOB Slow down. Listen. Offer support softly.

> *Slow down. Acknowledge without rushing. No cheerful language. Offer options softly. Transition to in-channel handoff if severity rises.*

VOICE *"Thanks for telling me that — it takes something to say. What feels right? None of these have to be the answer."*

Nudge

PROACTIVE · URGENCY

JOB Reach them about a deadline that matters specifically to them.

> *Open with disclosure (who you are, why). Be specific. Offer in-chat path. Respect adjust-preferences. Pause if distress hold is engaged.*

VOICE *"Hi Aisha — SSAF deferral deadline is in 6 days. I can walk you through the form in chat. About three minutes."*

Check-in

PROACTIVE · OPEN

JOB No agenda. Open warmly when a behavioural signal flags.

> *No agenda. Open warmly, no assumptions. Surface soft options. Transition to Express if "struggling". Close warmly without prying.*

VOICE *"Hey Mark — no agenda, just checking in. Anything getting in the way at the moment?"*

> *Cross-cutting concerns — brand voice, safety rails, identity, distress override — live in the always-in system prompt, not duplicated per pattern. The pattern prompt just adds the specialisation.*

Full production prompts live at `src/data/skills/<pattern>/prompts.md` — YAML frontmatter + system prompt body + routing rules + tool definitions + examples + safety overrides. Anthropic Skills folder format; interoperable with `~/.claude/skills/` and jcode.